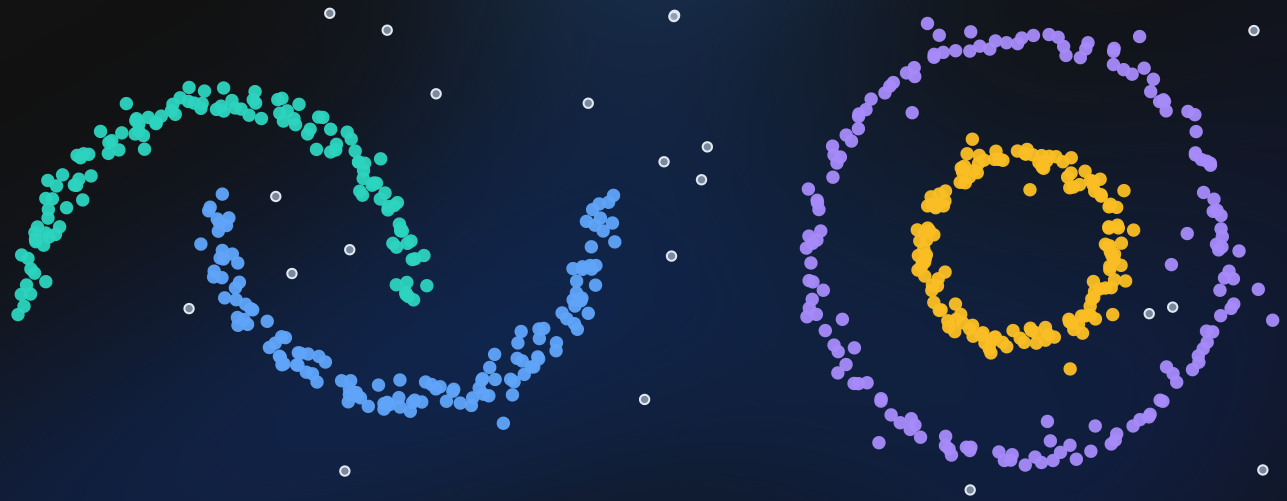
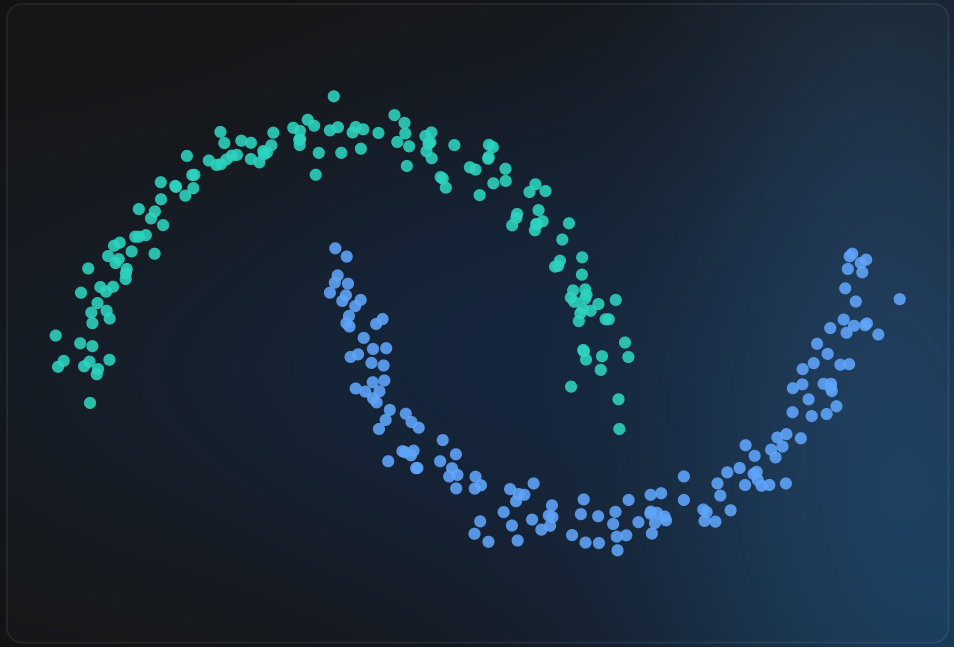
Hierarchical and Density-Based Clustering

Clusters without centroids



Unsupervised Learning · Foundations of Machine Learning

Where k-Means Left Off



DBSCAN · ARI 1.00

Centroid-based

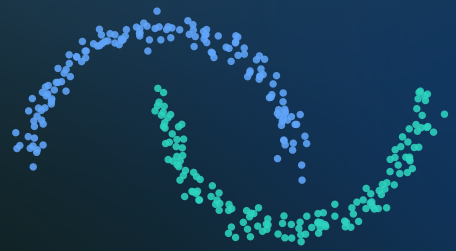
A cluster is the region closest to a center. Borders are straight, so curved shapes get cut.

Connectivity- and density-based

A cluster is a set of points **linked to each other** through close neighbors or dense regions. Any shape is allowed.

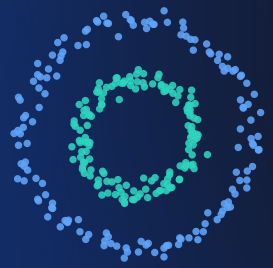
Two consequences: there is **no center** to summarize a cluster, and a point need not belong to any cluster at all. It can be **noise**.

Today: Three Ways to Cluster Without Centroids



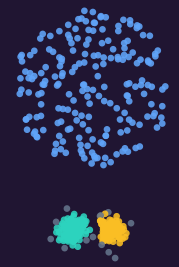
Hierarchical

Repeatedly merge the two closest clusters. The output is a **tree**; cut it to get any number of clusters.



DBSCAN

Grow clusters through **dense neighborhoods** of radius ε. Points in sparse regions become noise.



HDBSCAN

DBSCAN at **every ε at once**, keeping the clusters that persist. Handles clusters of different densities.

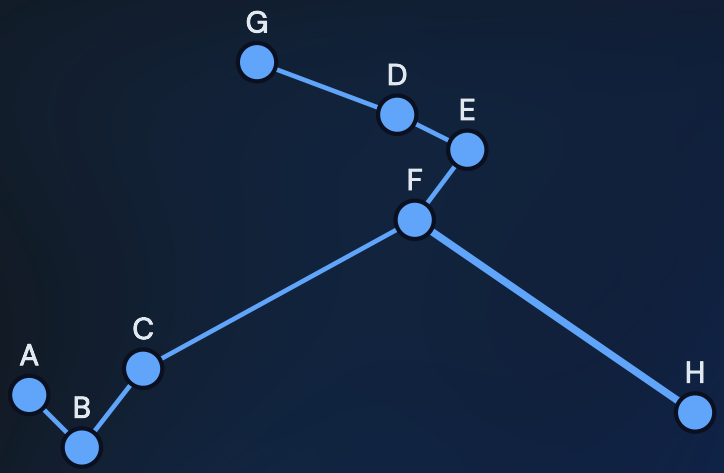
Then: three real datasets, and how to compare clusterings when there is no test accuracy.

Part 1 · Hierarchical Clustering

Build every clustering at once, as a tree

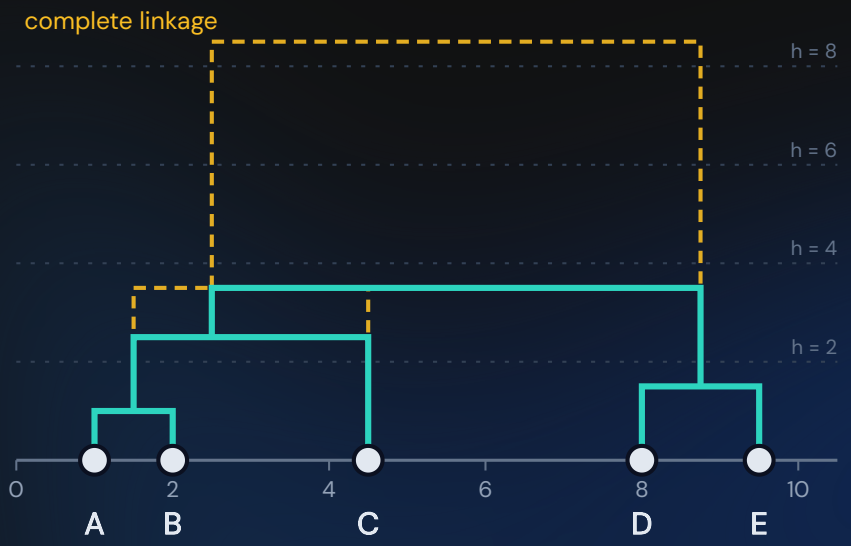
Agglomerative Clustering: Merge the Closest Pair

Start with every point alone · merge the two closest clusters · repeat until one cluster is left



Merge H + ABCDEFG at distance **3.88** · 1 cluster left

Worked Example: Single Linkage by Hand



Single linkage

Merge heights `1.0, 1.5, 2.5, 3.5`

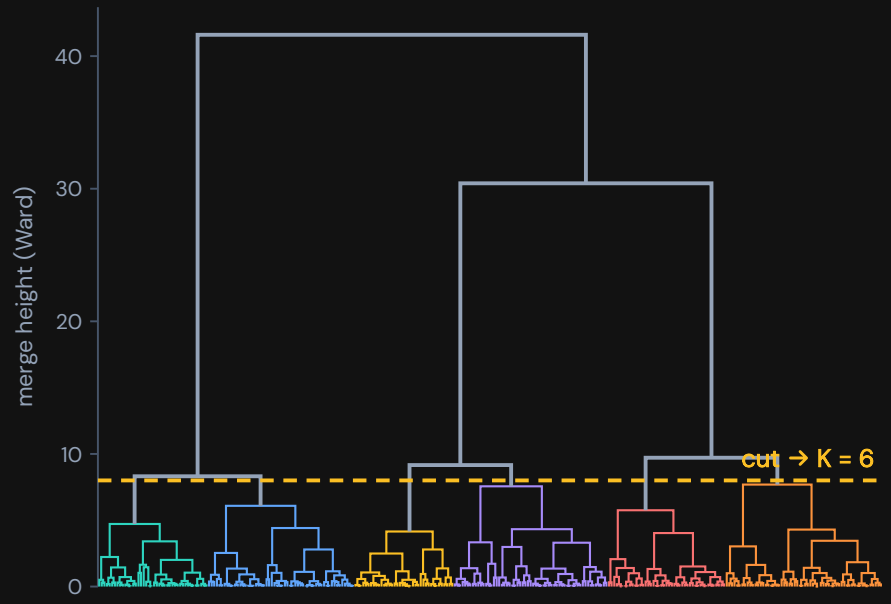
Complete linkage

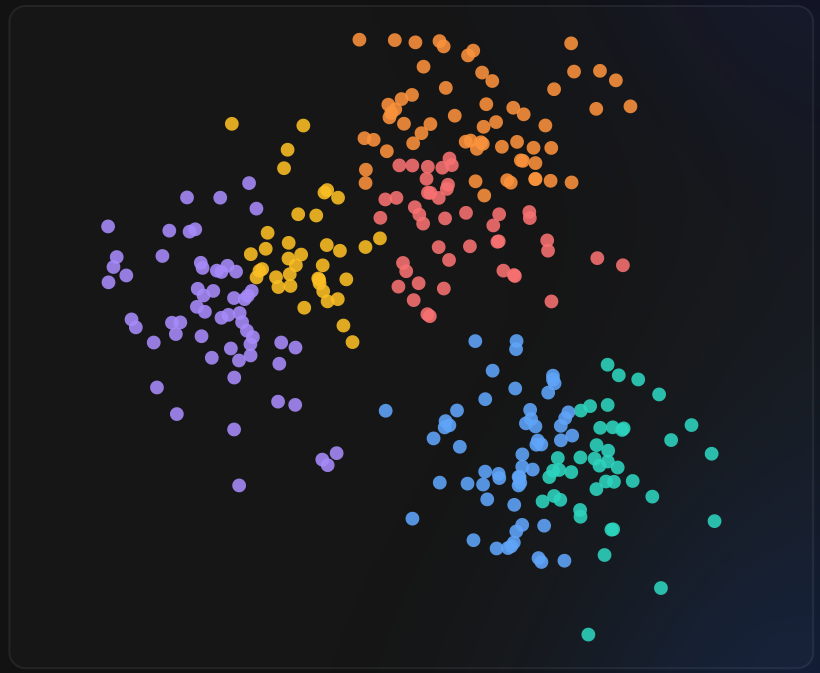
Merge heights `1.0, 1.5, 3.5, 8.5`

Same merges here, but each height is the farthest pair, so the last merge sits at 8.5 (A to E).

After merging A and B: $d(AB,\, C) = \min\{d(A, C),\, d(B, C)\}$ for single linkage, $\max\{\cdot\}$ for complete linkage.

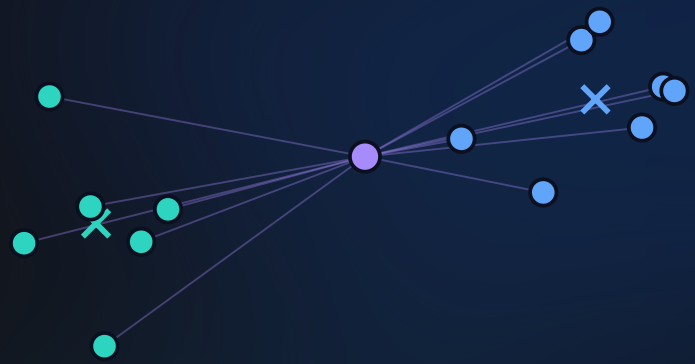
Reading a Dendrogram: Cut the Tree





Long vertical branches = groups that stay separate for a long time. Cut across the longest ones.

Linkage: How Far Apart Are Two Clusters?



Ward · increase in within-cluster squared error: 42.9

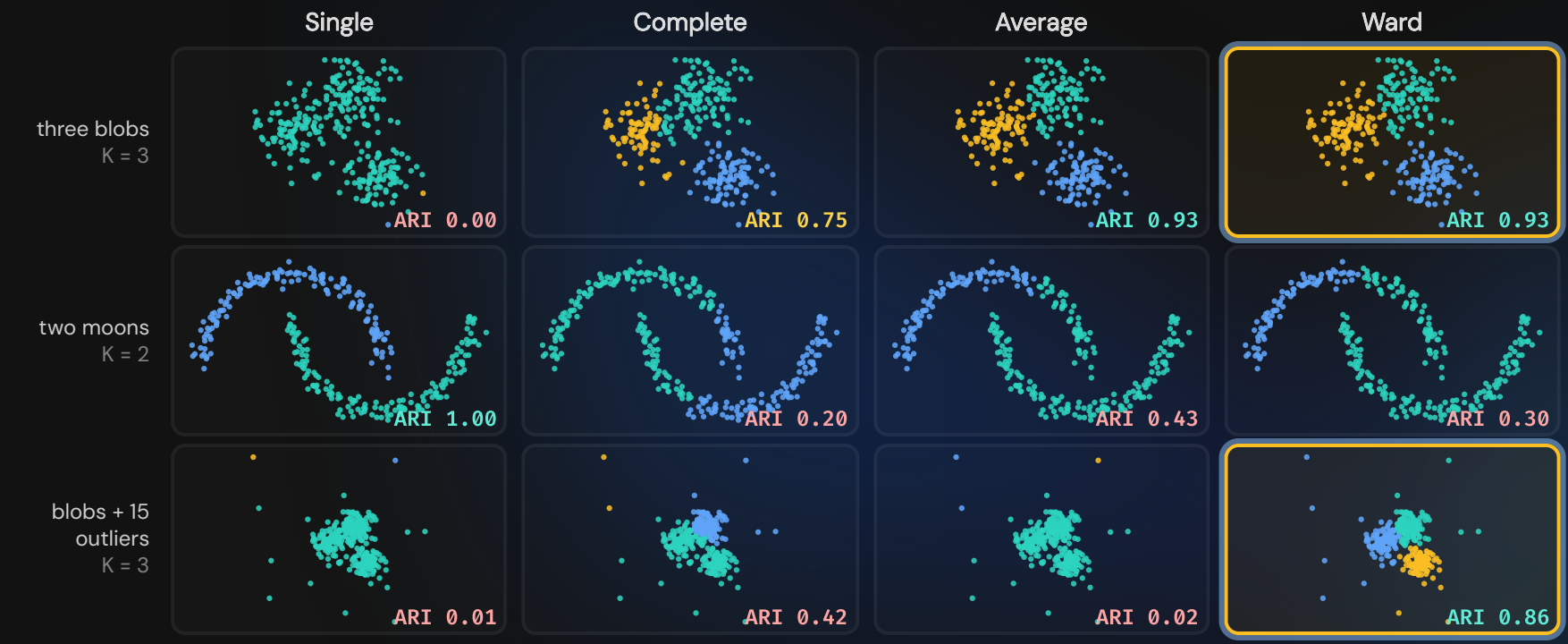
Single · closest pair $\min_{a \in A,\, b \in B} \|a - b\|$

Complete · farthest pair $\max_{a \in A,\, b \in B} \|a - b\|$

Average · all pairs $\frac{1}{|A||B|} \sum_{a \in A} \sum_{b \in B} \|a - b\|$

Ward · growth in squared error $\frac{|A||B|}{|A|+|B|} \|\mu_A - \mu_B\|^2$

Linkage Changes the Answer



Ward behaves like k-means: compact blobs, good on blobs, bad on moons.

Hierarchical Clustering in Code

```python
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import linkage, fcluster, dendrogram

# flat labels, like any scikit-learn clusterer
agg = AgglomerativeClustering(n_clusters=3, linkage="ward")
labels = agg.fit_predict(X)

# the whole tree: one row per merge
Z = linkage(X, method="ward")    # [child a, child b, height, size]
print(Z.shape)                   # (299, 4)
print(Z[-4:, 2].round(1))        # [ 9.2  9.7 30.4 41.6]

labels3 = fcluster(Z, t=3, criterion="maxclust")  # cut at K = 3
dendrogram(Z, truncate_mode="lastp", p=12)        # draw the top
```

scikit-learn gives labels for one K, or one cut height with `distance_threshold=`.

SciPy gives the full tree `Z` to plot and cut many ways. Both agree: here ARI between them is 1.0.

The last merge heights jump from **9.7** to **30.4**: a big gap, so K = 3 is the natural cut.

Cost and When to Use It

Memory for all pairwise distances (8-byte floats, n(n−1)/2 pairs)

Bar length on a log scale. Time is between $O(n^2)$ and $O(n^3)$ depending on the linkage.

Use it when

n is up to roughly ten thousand · the **tree itself** is the answer (taxonomies, gene families, document topics) · you want any distance, even a precomputed one · you don't know K yet

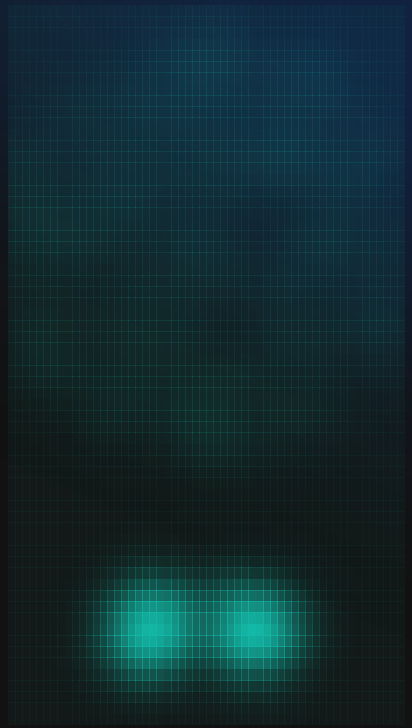
Watch out for

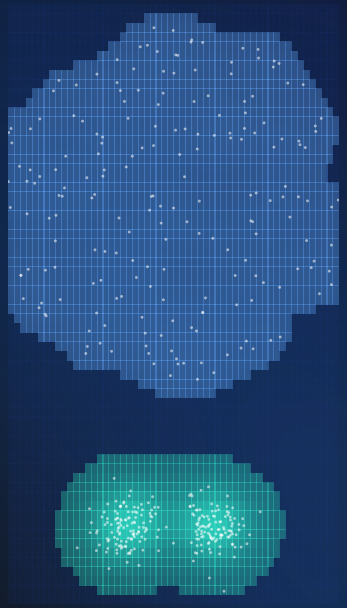
Quadratic memory · merges are greedy and never undone · single linkage chains, Ward assumes blobs

Part 2 · Density-Based Clustering

Clusters are dense regions, separated by sparse regions

Clusters Are Islands of Density





2 islands at sea level 0.025

DBSCAN: Core, Border, Noise

Two settings: a radius **ε = 1** and a count **minPts = 4** (the point itself counts).

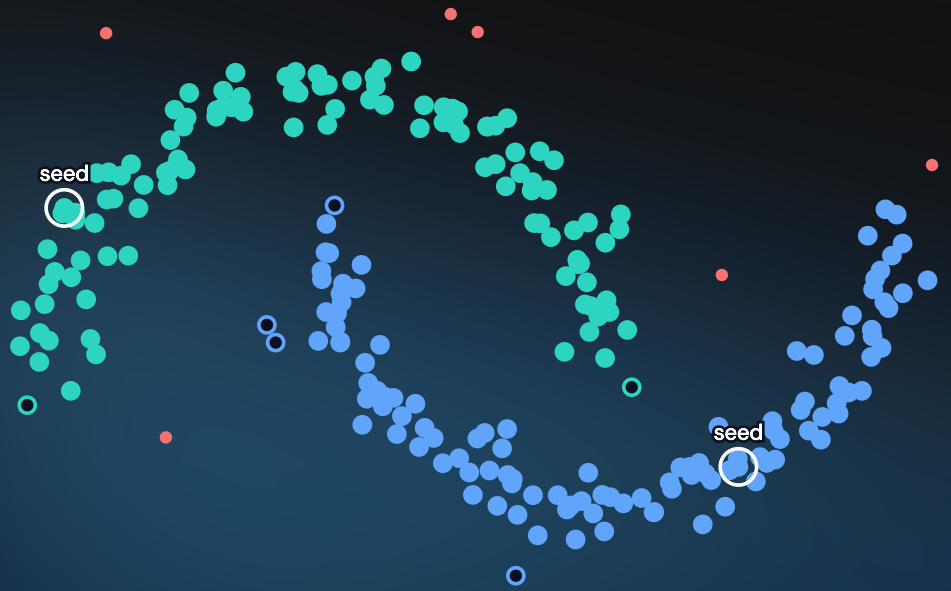
Core: at least minPts points within ε.

Border: not core, but within ε of a core point.

Noise: neither. Labeled −1.

Cluster: core points within ε of each other, chained together, plus their border points.

How DBSCAN Grows a Cluster



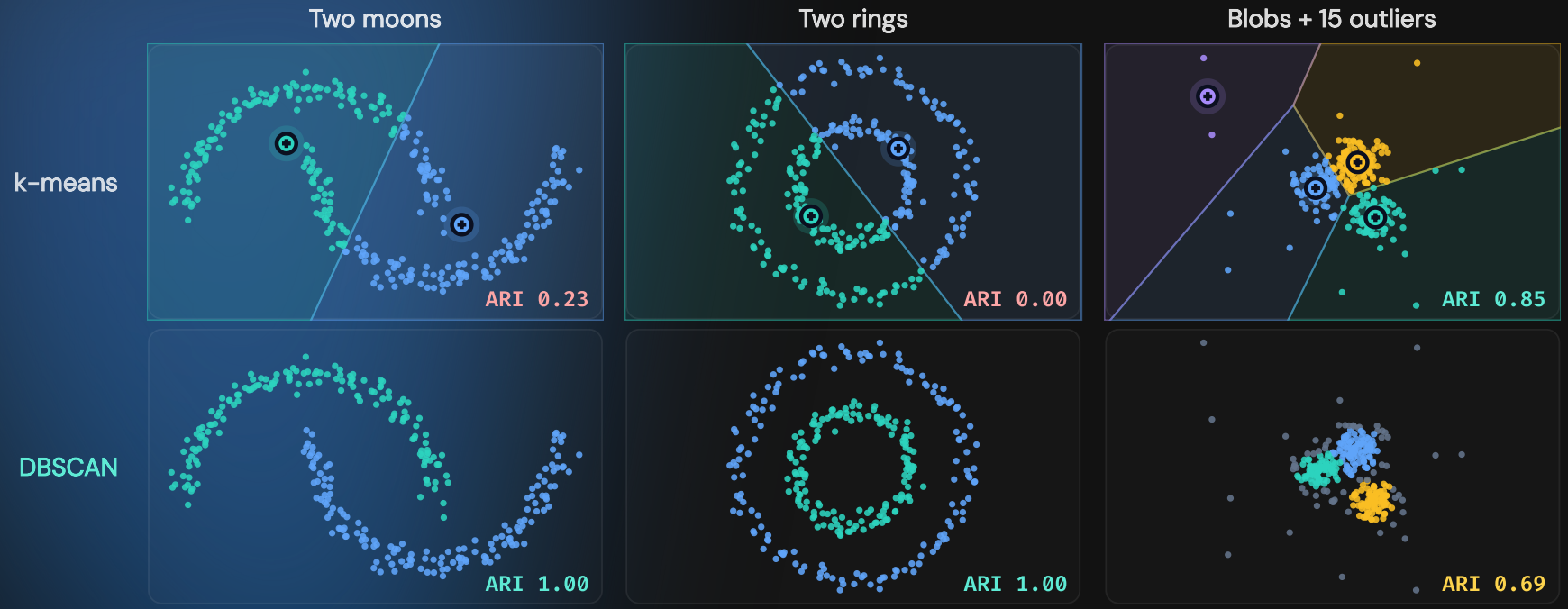
Cluster 1 112
points reached · wave 17 of 17

Cluster 2 116
points reached · wave 13 of 13

Noise 6
never within ε of a core point

core border

DBSCAN on k-Means' Failure Cases



ε = 0.2, minPts = 5
0 noise points; both shapes followed exactly

ε = 0.2, minPts = 5
0 noise points; both shapes followed exactly

ε = 0.55, minPts = 10
13 of 15 outliers are noise, plus 48 blob-edge points

DBSCAN in scikit-learn

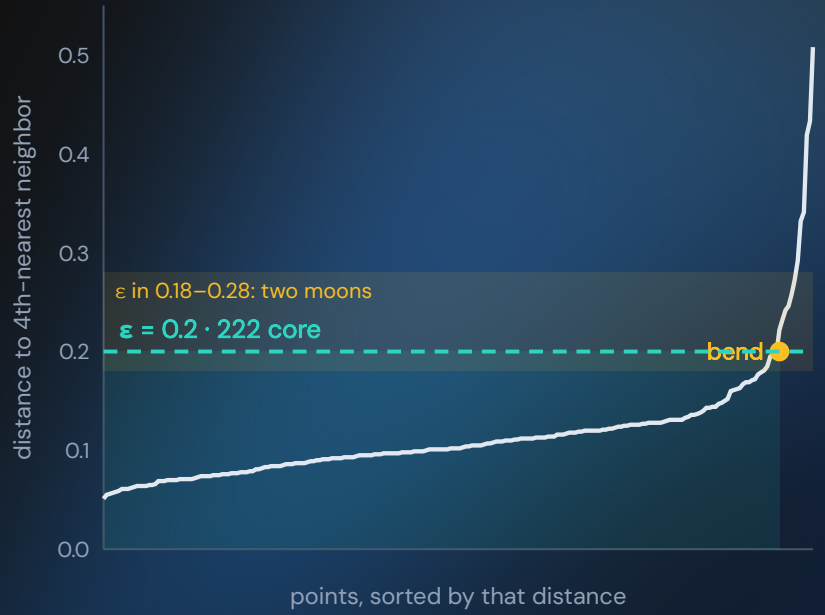
```
from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors

db = DBSCAN(eps=0.2, min_samples=5).fit(X)
labels = db.labels_                    # cluster ids, -1 = noise
print(labels.max() + 1,                # 2 clusters
      (labels == -1).sum(),            # 0 noise points
      len(db.core_sample_indices_))    # 299 core points

# choosing eps: distance to the 4th nearest other point
d, _ = NearestNeighbors(n_neighbors=5).fit(X).kneighbors(X)
kdist = np.sort(d[:, -1])              # plot it, look for the knee
```

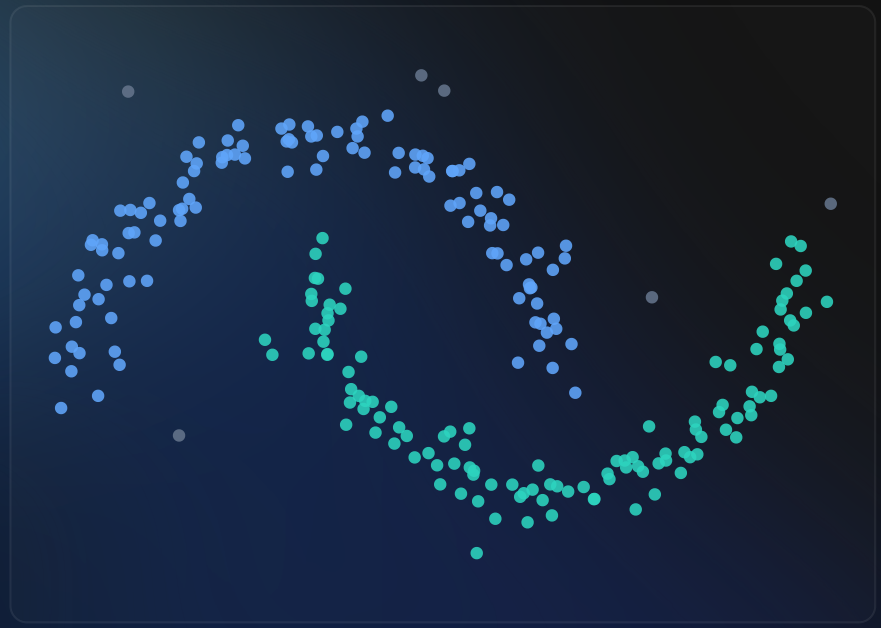
Scale first. ε is a distance in feature units. Use `make_pipeline(StandardScaler(), DBSCAN( ... ))`.

No predict(). New points need a refit, or assign them to the cluster of their nearest core point yourself.

Any metric. `metric="haversine"` for latitude and longitude, `"cosine"` for text embeddings, or `"precomputed"`.

Choosing ε: The k-Distance Plot

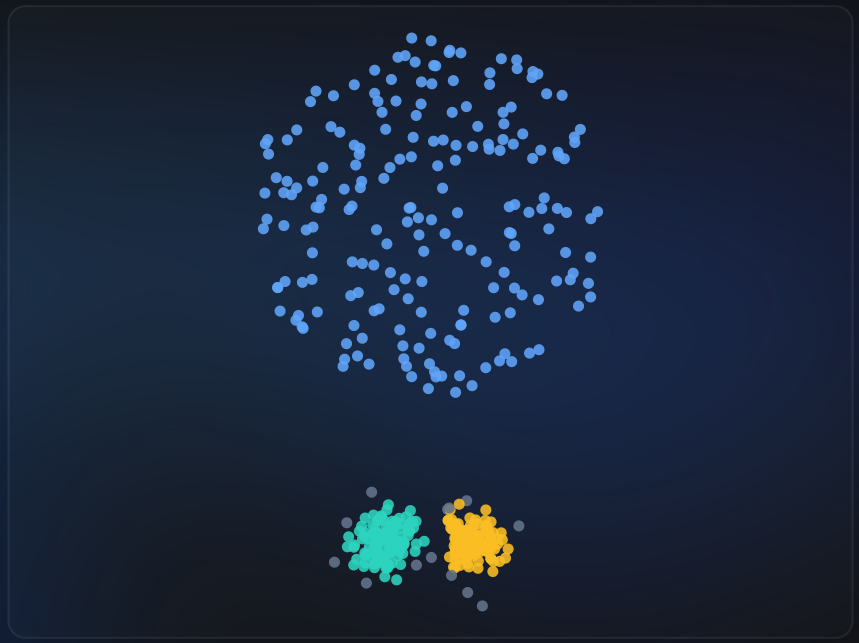




2 clusters · 6 noise

Just past the bend: the dense points fall below the line, the scattered ones rise sharply above it.

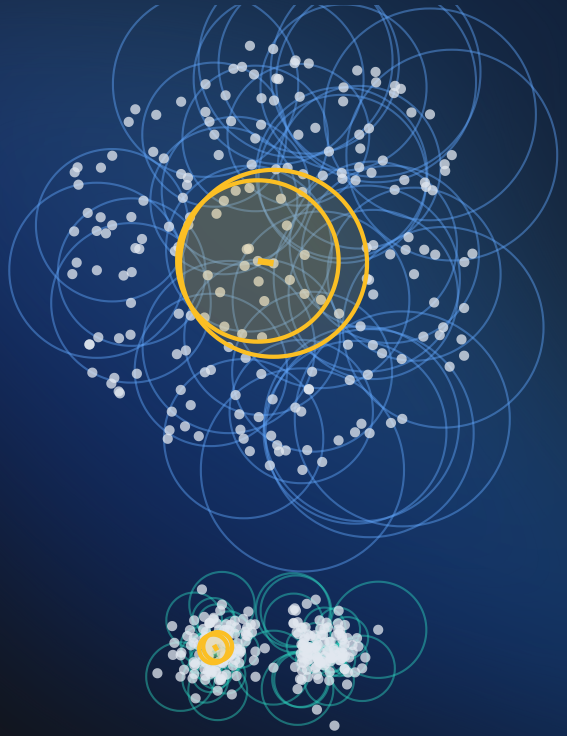
Where DBSCAN Fails: Clusters of Different Density



HDBSCAN · min_cluster_size = 25

HDBSCAN: all three groups, 13 noise points. ARI 0.96.

HDBSCAN, Step 1: Measure Density Point by Point



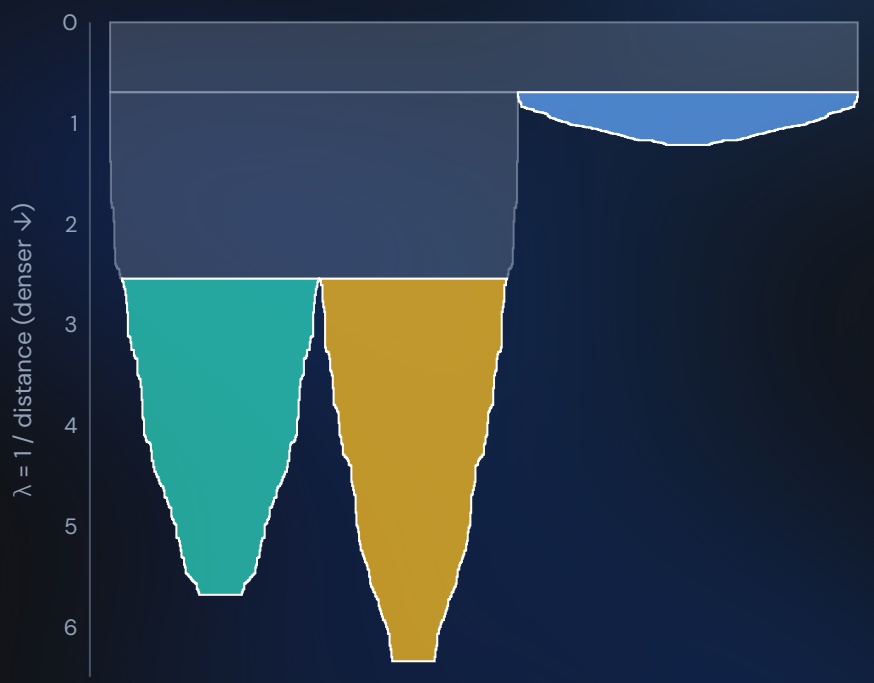
$$d_{\text{mreach}}(a, b) = \max\{\text{core}_k(a),\ \text{core}_k(b),\ d(a, b)\}$$

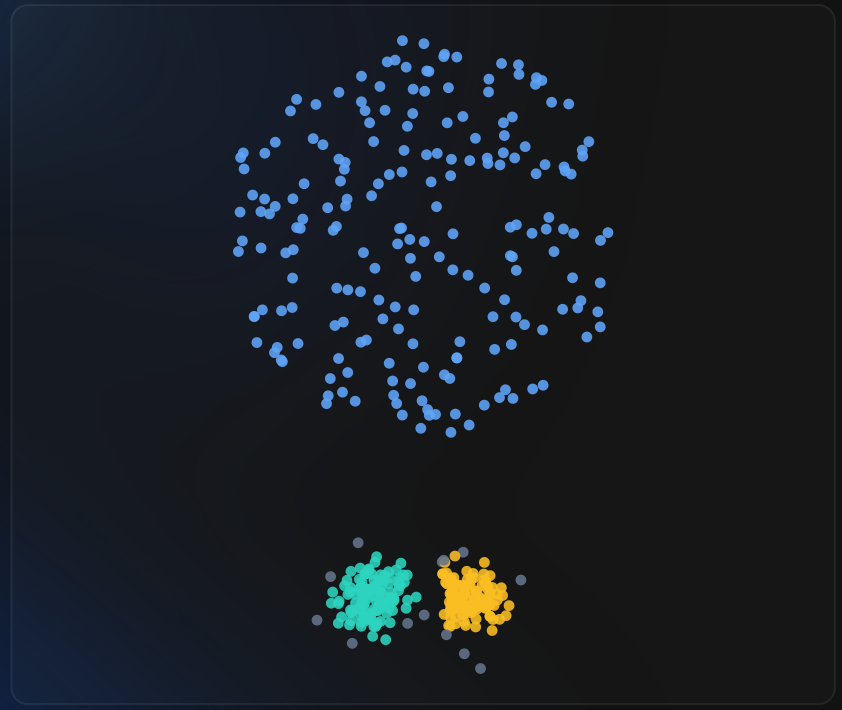
Core distance of a point: how far to its 25th-nearest neighbor. Small in dense blobs, large in the sparse disk.

	distance	mutual reach.
tight-blob pair	0.05	**0.16**
disk pair	0.17	**1.03**

Inside a dense blob the two numbers are close. In the sparse disk, distances are stretched up to the core distance, so sparse points look farther apart than they are.

HDBSCAN, Step 2: Keep the Clusters That Last





HDBSCAN keeps the branches that last longest, each at its own level.

HDBSCAN in scikit-learn

```python
from sklearn.cluster import HDBSCAN

hdb = HDBSCAN(min_cluster_size=25).fit(X)
labels = hdb.labels_            # -1 = noise
np.unique(labels, return_counts=True)
# (array([-1, 0, 1, 2]), array([ 13, 200, 117, 110]))

hdb.probabilities_              # strength of membership, 0 to 1
# a DBSCAN-style flat cut of the same tree:
hdb.dbscan_clustering(cut_distance=0.25, min_cluster_size=25)
```

min_cluster_size: the smallest group worth calling a cluster. The main setting.

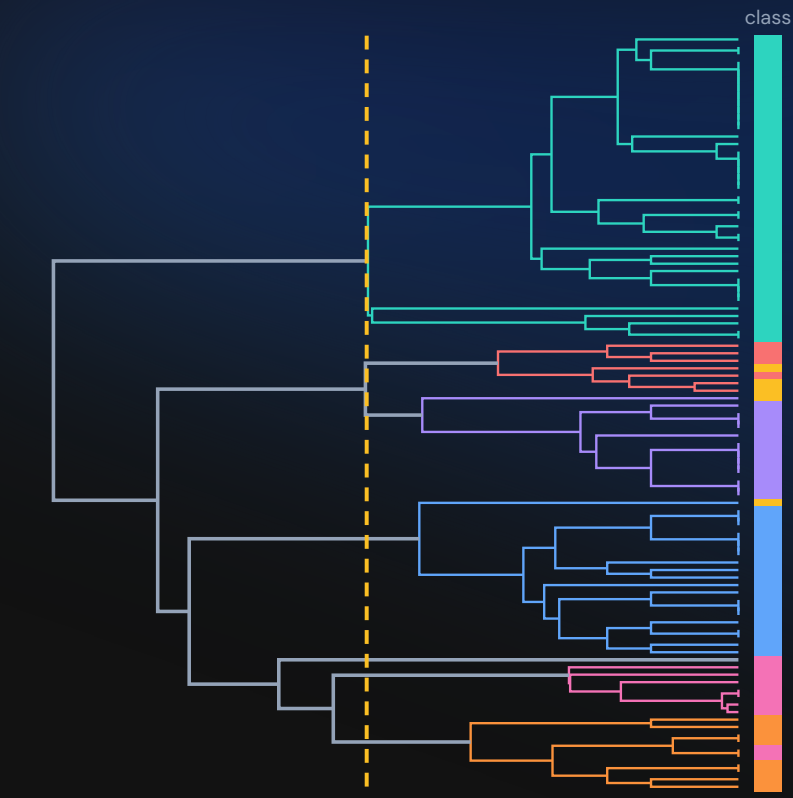
min_samples (defaults to min_cluster_size): the k in the core distance. Larger = more conservative, more noise.

probabilities_: points near a cluster's core score near 1, points at its fringe lower.

Part 3 · Real Data

An animal taxonomy, the world's earthquakes, and California's cities

Example 1: A Family Tree for 101 Animals



CUT INTO K = 7 · ARI VS. TRUE CLASS 0.95

41 animals · mostly **mammals** (100%)
mink, polecat, cheetah, deer, opossum, hamster

7 animals · largest class: **amphibians** (57%)
toad, frog, frog 2, pitviper, newt, slowworm

14 animals · mostly **fish** (93%)
seasnake, sole, seahorse, piranha, chub, catfish

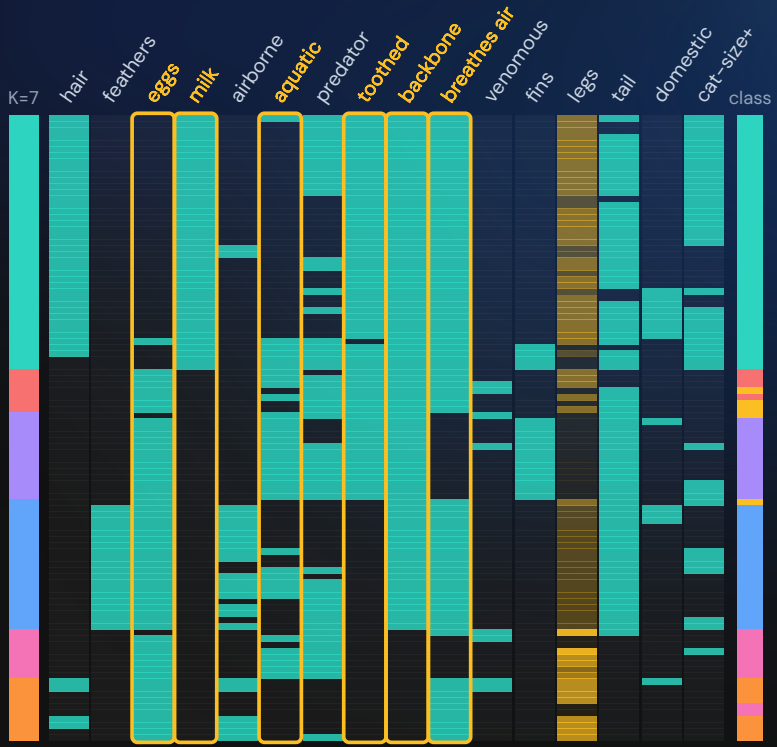
21 animals · mostly **birds** (95%)
tortoise, dove, lark, flamingo, duck, skimmer

1 animal · mostly **invertebrates** (100%)
scorpion

7 animals · mostly **invertebrates** (100%)
seawasp, clam, octopus, crayfish, lobster, crab

10 animals · mostly **insects** (80%)
honeybee, wasp, flea, termite, slug, worm

Zoo: Why These Groups?




```
zoo = pd.read_csv("zoo.data", header=None)
traits = zoo.iloc[:, 1:17].to_numpy(float)
traits[:, 12] /= 8          # legs 0-8 → 0-1
Z = linkage(traits, method="average",
            metric="cityblock")
groups = fcluster(Z, t=7, criterion="maxclust")
pd.crosstab(groups, zoo[17])   # column 17 = true type
```

The traits that differ most between the seven groups: **eggs, milk, aquatic, toothed, backbone, breathes**. Blocks of shared traits line up with the tree's branches.

Example 2: Earthquakes Trace the Plate Boundaries

Every M4.5+ earthquake in 2024 (USGS)

6,397 events → **60** clusters + **1,381** noise (22%)

DBSCAN, great-circle distance, ε = 150 km, minPts = 10

LARGEST CLUSTERS

Tonga	896
Philippines	680
Japan region	547
Vanuatu	366
Taiwan	314
Papua New Guinea	239

Earthquakes: k-Means vs. DBSCAN

DBSCAN: clusters follow each arc for thousands of kilometers; isolated events are noise.

Earthquakes in Code: Distance on a Sphere

```
eq = pd.read_csv("usgs_m45_2024.csv")
# haversine expects [latitude, longitude] in radians
X = np.radians(eq[["latitude", "longitude"]].to_numpy())
R = 6371.0                                    # Earth radius, km

db = DBSCAN(eps=150 / R, min_samples=10,
            metric="haversine", algorithm="ball_tree").fit(X)
eq["cluster"] = db.labels_
eq["cluster"].value_counts().head(4)
# -1    1381
#  1     896
#  4     680
#  0     547
```

Raw latitude/longitude is wrong: 179°E and 179°W are 358 apart in degrees but 220 km apart on Earth. Tonga–Fiji would be cut in two.

Haversine is great-circle distance on a unit sphere, so ε in km is ε / 6371. The k-distance plot works the same way.

Example 3: Where Are California's Cities?

HDBSCAN · 8 METRO AREAS

1	Los Angeles	7,527
2	SF Peninsula & San Jose	1,932
3	East Bay	1,590
4	San Diego	1,563
5	Sacramento	826
6	Fresno	817
7	Stockton & Modesto	754
8	Inland Empire	573
–	noise (rural, sparse)	5,058

block groups of 20,640

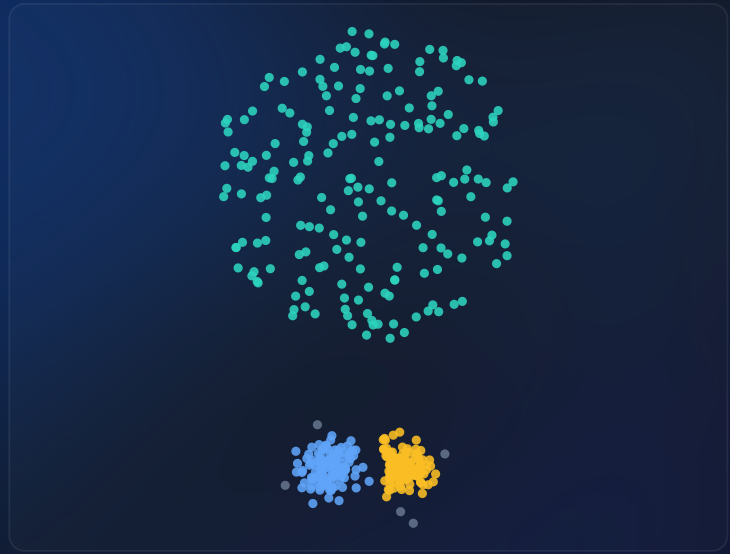
Part 4 · Comparing and Choosing

How do we know a clustering is any good?

Comparing Clusterings: ARI on Benchmarks

ARI: the fraction of point pairs that two clusterings treat the same way (same cluster in both, or different in both), adjusted so random labels score 0. **1** = identical, **0** = chance. Each method at its best settings; noise counts as singletons.

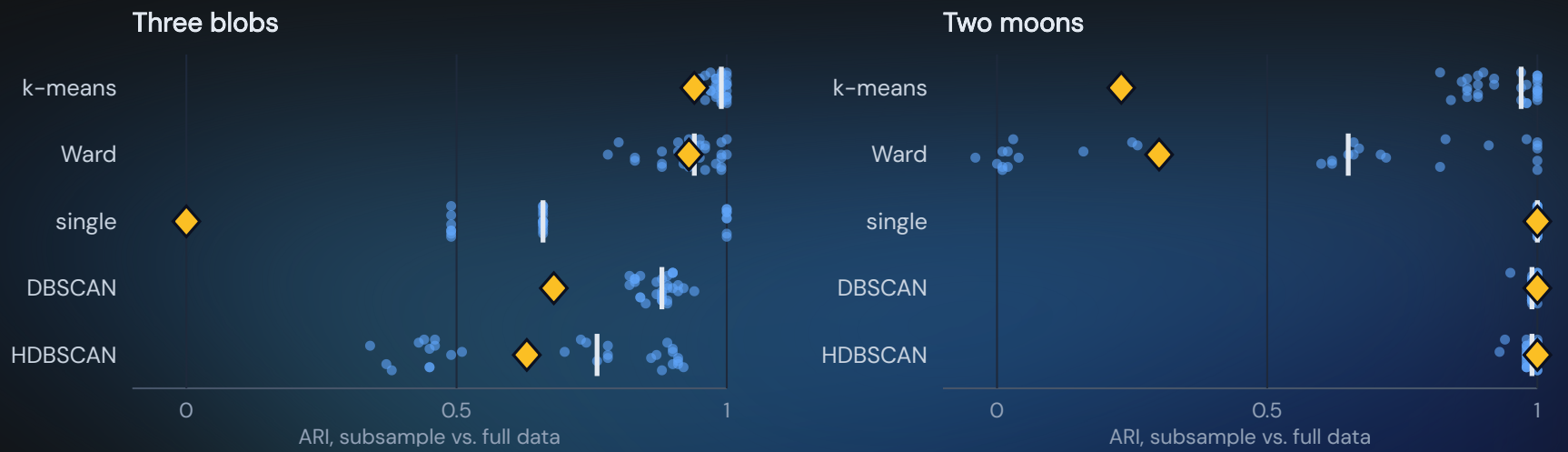
	k-means	Ward	single	DBSCAN	HDBSCAN
three blobs	0.94	0.93	0.00	0.68	0.63
two moons	0.23	0.30	1.00	1.00	1.00
two rings	0.00	0.01	1.00	1.00	1.00
blobs + outliers	0.84	0.86	0.01	0.69	0.63
two densities	0.46	0.46	0.66	0.70	0.97



HDBSCAN on two densities · ARI 0.97

Stability: Does the Answer Survive Resampling?

No labels needed: refit on 30 random 80% subsamples and compare each with the full-data clustering on the shared points.



◆ ARI vs. truth. Stable is not the same as right: k-means is stable on the moons, and wrong.

Choosing a Method

	CLUSTER SHAPE	NEEDS	NOISE?	SCALES TO	REACH FOR IT WHEN
k-means	round, similar size	K	no	millions	you need K prototypes or a partition of everything
Hierarchical	depends on linkage	linkage, cut	no	~10^4	the tree is the answer, or you have a custom distance
DBSCAN	any shape	ε, minPts	yes	millions	arbitrary shapes, similar density, outliers to flag
HDBSCAN	any shape	min_cluster_size	yes	~10^6	shapes **and** varying density; a good default for exploration
GMM (next)	ellipses	K, covariance	soft	large	overlapping clusters, probabilities of membership

Summary

Hierarchical

Merge the closest clusters until one is left. A dendrogram holds every K; cut at a big gap.

Linkage

Single follows chains (and chains through bridges); complete and Ward want compact blobs.

DBSCAN

Core, border, noise from ε and minPts. Any shape, flags outliers. Pick ε just past the k-distance bend.

HDBSCAN

Every ε at once; keeps the branches that last. Handles varying density with min_cluster_size.

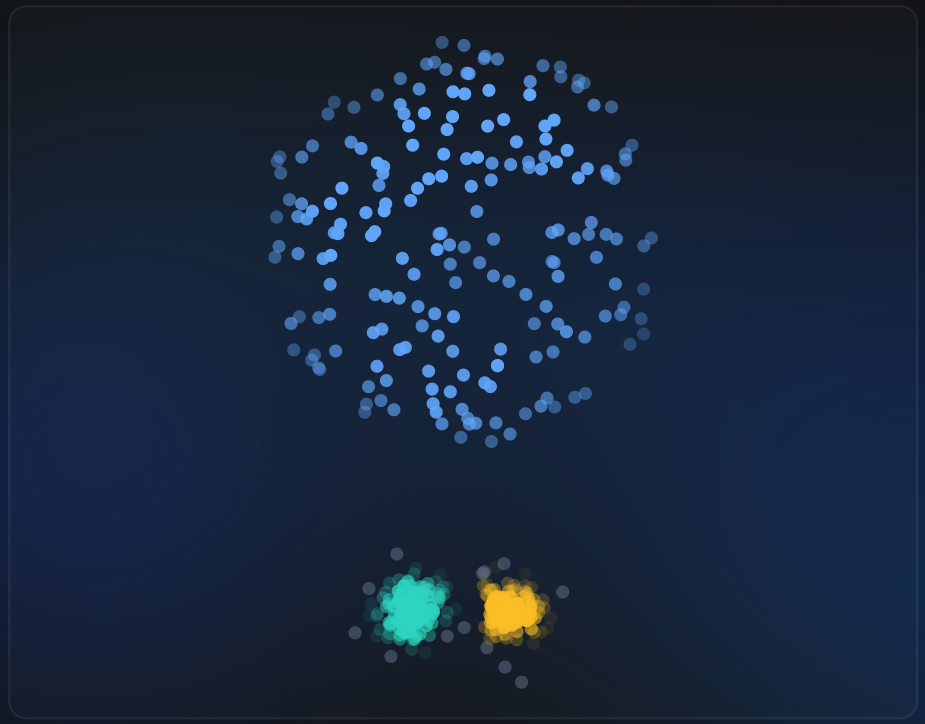
Distance is still the model

Scale features; use the metric that fits the data (Manhattan on traits, haversine on the globe).

Evaluate honestly

ARI only for benchmarks with known labels. Stability without labels, but stable ≠ right.

Next: Gaussian Mixture Models



So far: hard labels. Every point is in exactly one cluster, or noise, even at a fuzzy boundary.

Next deck: each cluster is a **Gaussian**, an ellipse with its own shape and tilt, and each point gets a **probability** of belonging to each cluster. Overlapping, stretched clusters become workable.